FisioMarca: mi parte del sistema

Guion de exposición para tablet · Auth · API Gateway · Carrito · Reservas · Eureka

Supuestos y puntos a verificar antes de imprimir

- **Stack:** Java 17, Spring Boot 4.1.1 y Spring Cloud 2025.1 (según los pom.xml y el README).
- **Base de datos:** MySQL 8.4 (docker-compose). El proyecto no usa PostgreSQL.
- **Seguridad:** JWT firmado con HS256 (librería JJWT). El login es propio. No usa el flujo OAuth2 Authorization Code con PKCE, como indica CONTRATO_FRONTEND.md.
- **Frontend:** no lo analicé. Asumo que el navegador guarda el token. Confirma con tu compañero dónde lo guarda (sección 2.4).
- **Discrepancia:** el README dice que Carrito llama a Reservas para convertir el carrito en reserva. En el código actual, Carrito solo llama a Auth y Catálogo. Asumo que el frontend llama directo a /reservas/checkout.
- **Logout:** el backend no tiene endpoint de cierre de sesión. Asumo que el frontend borra el token.

1. Resumen ejecutivo

LEER EN VOZ ALTA

Buenos días. Mi parte de FisioMarca controla quién entra al sistema y qué puede hacer cada persona.

Imaginen una clínica con un solo ingreso. En ese ingreso hay un guardia, que es el API Gateway. Nadie pasa sin mostrar una pulsera sellada: el token JWT. Esa pulsera la entrega el servicio de Autenticación, que verifica el correo y la contraseña, y si todo está bien emite el token.

Dentro del sistema, cada área tiene su propia puerta. El Carrito es donde el paciente arma su compra de sesiones. Reservas es la agenda, donde se eligen horarios y se paga la cita. Cada módulo revisa que el paciente solo toque sus propios datos.

Y para que el Gateway sepa a dónde enviar cada petición, usamos Eureka, que funciona como el directorio de un centro comercial. Cada servicio se registra ahí y los demás lo encuentran sin direcciones fijas.

En resumen: identidad, control de acceso y enrutamiento. El paciente navega de forma segura, y el personal tiene herramientas de administración separadas.

2. Flujo completo con énfasis en seguridad

2.1 Paso a paso, de abrir la app a cerrar sesión

1. El usuario abre la app. Todavía no hay token, así que solo ve el login.
2. Envía **POST /auth/login** con correo y contraseña. El Gateway lo deja pasar porque es una ruta pública.
3. Auth busca al usuario y compara la contraseña con su **hash BCrypt**. Nunca compara texto plano.
4. Auth genera el **JWT** con el correo, los roles, el id de usuario, el nombre, el id de cliente y la expiración (1 hora). Lo firma con la clave secreta.
5. El frontend guarda el token y lo envía en cada petición: **Authorization: Bearer <token>**.
6. El Gateway valida la **firma** y la **fecha de vencimiento**. Si falla, responde 401. Si pasa, busca en Eureka el servicio destino y reenvía la petición.
7. El microservicio vuelve a validar el token como *Resource Server*, lee los roles y revisa que el recurso sea del paciente.

8. Si todo cuadra, responde con los datos. Si no, devuelve 403.
9. Para cerrar sesión, el frontend borra el token. El backend no tiene logout. El token sigue siendo válido hasta vencer, por eso es una limitación que se menciona (sección 4).

2.2 ¿Cómo se valida la identidad? (Auth)

Auth es la única oficina que emite tokens. Usa Spring Security con un DaoAuthenticationProvider: busca al usuario por correo y compara la contraseña con BCrypt, un algoritmo de hash que no se puede revertir. Por eso la base de datos nunca guarda contraseñas en texto plano. Además, el servidor no guarda sesiones (política STATELESS): toda la información de quién eres viaja dentro del token.

> **Analogía**
>
> Auth es la oficina que emite pulseras. Revisa tu documento y te entrega una pulsera con un sello que solo ella puede poner.

2.3 ¿Cómo se protegen las rutas? (Gateway + JWT)

El Gateway tiene un filtro global, *JwtAuthenticationFilter*, que se ejecuta antes que cualquier ruta. Deja pasar sin token: /auth/login, /auth/register, /usuarios/registrar, el catálogo en modo lectura (GET /catalogo/**) y el servicio SOAP (/ws/**). Todo lo demás exige un token válido.

Detalle importante: el Gateway **no revisa roles**. Solo confirma que el token es auténtico y que no venció.

2.4 ¿Dónde se guardan los tokens?

El backend no guarda tokens, porque no tiene sesiones. Quien los guarda es el frontend, en el navegador. Respuesta corta: **“En el navegador del paciente, en el lugar que definió el frontend. El servidor no lo almacena.”**

> **Saber por si preguntan (no leer)**
>
> - **Memoria de la app:** más seguro frente a robos por JavaScript, pero se pierde al recargar la página.
> - **localStorage:** cómodo, pero cualquier script inyectado (XSS) puede leerlo.
> - **Cookie HttpOnly:** JavaScript no puede leerla. Requiere protección contra CSRF.

2.5 ¿Qué pasa si alguien intercepta la comunicación?

Un JWT está **firmado, no cifrado**. Cualquiera puede leer su contenido, porque es solo Base64. Por eso nunca se guarda información sensible dentro. La firma sí impide alterarlo: si alguien cambia su rol de CLIENTE a ADMINISTRADOR, la firma ya no coincide y el token se rechaza.

El riesgo real es que roben el token completo y lo usen hasta que venza (1 hora). Las defensas son: **HTTPS** para cifrar el canal, expiración corta y revocación. En la demo local se usa HTTP, y eso debes reconocerlo.

2.6 ¿Cómo sabe el Gateway que el usuario tiene permiso?

No lo sabe. El Gateway solo confirma identidad. El permiso lo decide el microservicio dueño del recurso, leyendo el claim *roles*. Por ejemplo, GET /reservas/lista exige ROLE_ADMINISTRADOR. Un token de paciente pasa el Gateway y Reservas responde 403. Son dos capas de revisión.

2.7 ¿Cómo ayuda Eureka a que la comunicación sea segura?

Eureka **no aporta cifrado ni autenticación**. Aporta descubrimiento. La seguridad viene de tres cosas: (1) solo el Gateway se publica al exterior, mientras que los microservicios viven en la red interna de Docker; (2) cada servicio valida el token; (3) las rutas se definen en un solo lugar, el Gateway.

3. Guion de exposición por módulo

3.1 Auth (autenticación)

> **LEER EN VOZ ALTA**
>
> Auth es el módulo que dice quién eres. Recibe tu correo y tu contraseña y, si son correctos, te entrega un token. Ese token es tu credencial sellada: lo presentas en cada petición y no tienes que escribir la contraseña otra vez.

Explicación simple: es la recepción de la clínica. Verifica tu identidad y te entrega una credencial.

Preguntas probables y respuestas cortas:

¿Cómo guardan las contraseñas?

Con BCrypt. Solo se guarda el hash; el texto original nunca se almacena.

¿Qué lleva el token?

El correo como sujeto, los roles, el id de usuario, el nombre, el id de cliente y la fecha de vencimiento.

¿Por qué no usan sesiones?

Una sesión obliga al servidor a recordar a cada usuario. Con JWT, cada petición se valida sola y el servicio escala mejor.

Saber por si preguntan (no leer)

- Spring Security: DaoAuthenticationProvider compara la contraseña con UserDetailsService.
- /auth/** es público. /usuarios/** exige token. /usuarios/lista exige ADMINISTRADOR.

3.2 API Gateway

LEER EN VOZ ALTA

El Gateway es la única puerta pública del sistema. Todas las peticiones entran por aquí. Revisa el token y, si es válido, reenvía la petición al microservicio correcto. Los demás servicios no están expuestos a internet.

Analogía

Es el guardia de la entrada del centro comercial. Revisa tu pulsera y te indica a qué tienda ir.

Preguntas probables y respuestas cortas:

¿Qué hace su filtro?

Verifica la firma y la expiración del token. No revisa roles.

¿Por qué un Gateway y no que cada servicio valide solo?

Un solo punto de control, rutas centralizadas y servicios que no se exponen. Cada microservicio igual valida el token.

¿Qué es CORS aquí?

Permite al navegador llamar al Gateway solo desde los orígenes del frontend, con credenciales.

Saber por si preguntan (no leer)

- Spring Cloud Gateway (WebFlux, reactivo). Cada ruta usa Path=. La URI lb://NOMBRE consulta Eureka y balancea la carga.
- El filtro tiene orden -100, así que se ejecuta primero. Si el token falla, responde 401 en JSON.

3.3 Carrito de compras

LEER EN VOZ ALTA

El Carrito guarda los servicios que el paciente quiere comprar antes de pagar. Cada paciente tiene su propio carrito. Antes de cambiar algo, el sistema verifica que el carrito le pertenece. Y el precio siempre se toma del Catálogo, no de lo que envía el navegador.

Analogía

Es la bolsa de compras del paciente. Solo él puede tocarla, y los precios los pone la tienda, no el cliente.

Preguntas probables y respuestas cortas:

¿Puede un paciente ver el carrito de otro?

No. AccesoCliente compara el clienteId del token con el de la ruta. Solo pasan el dueño del carrito y el ADMINISTRADOR.

¿Y si el paciente manda un precio falso?

El precio no se acepta del cliente. Se consulta a Catálogo por Feign y se guarda ese valor.

¿Por qué el límite de 4 sesiones?

Es una regla de negocio validada en el servidor, no solo en el frontend.

Saber por si preguntan (no leer)

- Feign llama a AUTH (existe el cliente) y a MICRO-CATALOGO (precio) usando el nombre del servicio, no una IP.
- Esas llamadas internas no reenvían el token. Es una limitación; ver sección 4.

3.4 Reservas (citas)

LEER EN VOZ ALTA

Reservas es la agenda de la clínica. Aquí se revisan horarios, fisioterapeutas y salas disponibles. El paciente aparta su cita y tiene 10 minutos para pagar; si no paga, el horario se libera. Solo el administrador puede registrar citas presenciales, atender y reasignar.

Analogía

Es el calendario compartido de la clínica. Si una cita choca con otra, el sistema la rechaza antes de guardarla.

Preguntas probables y respuestas cortas:

¿Cómo evitan dos citas en el mismo horario?

Antes de guardar, buscan citas que se crucen con el mismo fisioterapeuta o sala. Si hay cruce, rechazan la reserva. Atienden de 07:00 a 12:00 y de 14:00 a 19:00, y los domingos cierran.

¿Qué pasa si no pagan?

La retención de 10 minutos vence y un proceso libera el horario.

¿Los pagos son reales?

No. Son simulados (tarjeta, celular o código). No hay pasarela de pago real, y eso se dice con honestidad.

Saber por si preguntan (no leer)

- Reglas por ruta: registrar, atender, check-in, reasignar, lista y dashboard exigen ADMINISTRADOR.
- Cada acción del paciente (checkout, pagar, cancelar) valida que la orden sea suya con AccesoCliente.

3.5 Eureka (service discovery)

LEER EN VOZ ALTA

Eureka es el directorio de servicios. Cada microservicio, al arrancar, se registra con su nombre y su dirección. Cuando el Gateway necesita llegar a Reservas, pregunta a Eureka dónde está. Si un servicio cambia de dirección o se agregan copias, nada se rompe.

Analogía

Es el directorio del centro comercial: indica en qué local está cada tienda, aunque se mude.

Preguntas probables y respuestas cortas:

¿Eureka protege el sistema?

No. Es descubrimiento, no seguridad. La seguridad está en el token, en las validaciones de cada servicio y en la red interna.

¿Qué es el load balancing aquí?

Si hay varias instancias con el mismo nombre, Spring Cloud LoadBalancer reparte las peticiones. En la demo hay una instancia por servicio, así que es una capacidad, no algo visible.

¿Cómo se entera de un cambio?

Cada servicio envía latidos cada 10 segundos. En la configuración de Docker, el cliente consulta el registro cada 5 segundos.

Saber por si preguntan (no leer)

- @EnableEurekaServer en el puerto 8761. Feign y lb:// usan nombres lógicos como AUTH-SERVICE y MICRO-CARRITO.
- El panel de Eureka no tiene login y el compose lo publica en el 8761. Esto se reconoce como limitación.

4. Limitaciones que conviene reconocer

Si el profesor pregunta “¿es seguro?”, la mejor respuesta es reconocer estos puntos y explicar cómo se resolverían. Van de mayor a menor impacto.

4.1 Secreto JWT con valor por defecto

El valor de JWT_SECRET está escrito en application.yml y en docker-compose como respaldo. Quien tenga el repositorio puede firmar tokens válidos con cualquier rol, incluido ADMINISTRADOR, y todos los servicios los aceptarían.

Solución: una variable de entorno sin valor por defecto, un gestor de secretos y rotación periódica de la clave.

4.2 Tokens que no se pueden revocar

El logout solo borra el token en el navegador. Si alguien copió el token, sigue funcionando hasta que venza, a la 1 hora. Un cambio de rol tampoco surte efecto hasta el siguiente login, porque el rol viaja dentro del token.

Solución: una lista de revocación (por ejemplo en Redis) o tokens de vida corta con refresh token.

4.3 HTTP sin cifrar

En la demo, el tráfico viaja sin cifrar. Cualquiera que esté en la misma red puede leer el token de una petición.

Solución: TLS (HTTPS) en el Gateway, con certificado, y redirigir HTTP a HTTPS.

4.4 Llamadas internas sin token (Feign)

Cuatro llamadas entre microservicios no llevan token: Carrito y Reservas consultan a AUTH si el cliente existe (*/usuarios/clientes/existe/{id}*), y ambos consultan a Catálogo el precio (*/catalogo/servicios/buscar/{id}*). Ningún cliente Feign reenvía el encabezado *Authorization*, porque no hay un RequestInterceptor. Funcionan porque esos endpoints son públicos: en AUTH, *existe* tiene permitAll, y en Catálogo el GET es público.

Consecuencia: dentro de la red de Docker, cualquier servicio, o alguien que entre a esa red, puede consultar esos datos sin credenciales. Si mañana se protegieran esos endpoints, estas llamadas fallarían con 401.

Solución: un RequestInterceptor de Feign que copie el header Authorization de la petición entrante, o un token propio para la comunicación entre servicios.

4.5 Eureka sin protección

El panel y la API de Eureka en el puerto 8761 no piden login, y docker-compose lo publica en el host. Cualquiera que llegue a ese puerto puede ver qué servicios existen. Peor aún, podría registrar una instancia falsa con el nombre de un servicio y recibir tráfico que el Gateway reenvía por lb://.

Solución: autenticación básica en Eureka, no publicar su puerto y dejarlo solo en una red privada.

4.6 Pagos simulados

El pago de Reservas valida formatos de tarjeta, celular y código, pero no hay una pasarela real que confirme el cobro.

Solución: integrar una pasarela de pagos y no guardar nunca datos completos de tarjeta en la base de datos.

> **LEER EN VOZ ALTA**
>
> Es un proyecto académico. Sé qué medidas faltan para producción y cómo las implementaría.

5. Cheat sheet (una página)

Término	Qué es (≤10 palabras)
JWT	Token firmado que demuestra quién eres.
Claims	Datos dentro del token: usuario, roles, vencimiento.
Firma HS256	Sello con clave secreta; detecta cualquier alteración.
Bearer <token>	Formato del encabezado Authorization.
OAuth2	Marco de autorización con terceros; este proyecto no lo usa.
Resource Server	Servicio que exige token válido en cada petición.
BCrypt	Hash irreversible para guardar contraseñas de forma segura.
Stateless	Sin sesiones en el servidor; cada petición trae su token.
HTTPS	HTTP cifrado; evita leer tokens interceptados en tránsito.
API Gateway	Puerta única que reenvía peticiones a cada microservicio.
Enrutamiento	Regla que decide a qué servicio va cada ruta.
Service discovery	Registro automático de dónde está cada servicio.
Eureka	Servidor de service discovery de Netflix, usado aquí.
Load balancing	Reparte peticiones entre instancias del mismo servicio.
Feign	Cliente HTTP declarativo para llamar a otro microservicio.
CORS	Permiso del navegador para llamar a otro origen.
401 / 403	Sin identidad (401) o sin permiso (403).